

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH



**BÁO CÁO BÀI TẬP LỚN
ĐẠI SỐ TUYẾN TÍNH**

Đề tài 10

**Phân tích Randomized SVD (Truncated SVD)
và ứng dụng trong nén dữ liệu
theo tỉ lệ năng lượng cho trước**

Giảng viên hướng dẫn: Nguyễn Hữu Hiệp

Nhóm L05 – Danh sách thành viên

2213539	Phạm Quốc Toàn
2213496	Nguyễn Quốc Tín
2213299	Nguyễn Trường Thịnh
2213289	Nguyễn Hữu Thịnh
2213228	Hồ Viết Thiên
2213120	Trương Hữu Thái
2213080	Trương Hồng Tấn
2213063	Nguyễn Trung Tân
2212922	Nguyễn Quang Sáng

Thành phố Hồ Chí Minh, tháng 5/2026



Content

1 Mở đầu

1.1 Lý do chọn đề tài

Trong kỷ nguyên dữ liệu lớn, kích thước các ma trận biểu diễn dữ liệu thực tế (ảnh số, ma trận đánh giá, dữ liệu cảm biến, văn bản, ...) thường vượt xa khả năng lưu trữ và xử lý trực tiếp. Phân tích giá trị kỳ dị (Singular Value Decomposition – SVD) là một trong những công cụ trung tâm của đại số tuyến tính hiện đại: nó cho phép tách một ma trận bất kỳ thành tích của ba ma trận có cấu trúc đẹp, từ đó tạo ra cơ sở toán học cho hàng loạt ứng dụng như nén dữ liệu, giảm chiều, khử nhiễu, hệ thống gợi ý và xử lý ảnh.

Tuy nhiên, thuật toán SVD cổ điển (thông qua phương pháp Golub–Kahan hay Jacobi) có độ phức tạp $\mathcal{O}(mn \min(m, n))$ và đòi hỏi truy xuất toàn bộ ma trận nhiều lần. Khi m, n ở mức 10^5 trở lên, cách tiếp cận này trở nên bất khả thi cả về thời gian lẫn bộ nhớ. **Randomized SVD**, được Halko, Martinsson và Tropp hệ thống hóa năm 2011, là một cách tiếp cận đột phá: dùng phép chiếu ngẫu nhiên để rút trích nhanh một không gian con xấp xỉ tốt cho k giá trị kỳ dị lớn nhất, sau đó chỉ chạy SVD cổ điển trên một ma trận nhỏ hơn nhiều. Phương pháp này có cơ sở toán học vững chắc, sai số kiểm soát được và dễ song song hóa.

Đề tài tập trung vào Randomized SVD vì ba lý do:

- **Tầm quan trọng toán học:** đây là một kết quả tiêu biểu của đại số tuyến tính số (numerical linear algebra) hiện đại, kết hợp xác suất, hình học và giải tích ma trận.
- **Lĩnh vực áp dụng rộng:** học máy, xử lý ảnh, hệ thống gợi ý, nén tín hiệu, phân tích văn bản (LSA), tin sinh học.
- **Vấn đề thực tế cụ thể:** bài toán giảm dung lượng lưu trữ và rút gọn chiều biểu diễn cho dữ liệu lớn, sao cho vẫn giữ được một tỉ lệ năng lượng (thông tin) η cho trước.

1.2 Mục tiêu của đề tài

Báo cáo hướng tới các kết quả cụ thể sau:

- Trình bày được cơ sở lý thuyết của SVD đầy đủ, Compact SVD và Randomized SVD; nêu rõ ý nghĩa toán học của từng đại lượng.
- Xây dựng được tiêu chuẩn chọn k theo tỉ lệ năng lượng

$$\frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^r \sigma_i^2} \geq \eta,$$

và phân tích ý nghĩa của tham số η .

- Cài đặt thuật toán Randomized SVD và áp dụng cho một ma trận dữ liệu thực tế có kích thước lớn (trên 10^5 phần tử).
- Tái dựng ma trận xấp xỉ hạng thấp A_k , lưu trữ dưới dạng dữ liệu nén (U_k, Σ_k, V_k) và đánh giá sai số xấp xỉ $\|A - A_k\|_F$ theo chuẩn Frobenius.
- Phân tích mối quan hệ giữa tỉ lệ năng lượng giữ lại, sai số xấp xỉ và dung lượng lưu trữ; rút ra điều kiện thực tế để Randomized SVD đạt hiệu quả nén tốt.

1.3 Phạm vi nghiên cứu

- **Đối tượng nghiên cứu:** ma trận thực $A \in \mathbb{M}_{m \times n}(\mathbb{R})$ với $m \cdot n \geq 10^5$. Trong báo cáo, ma trận thí nghiệm là ma trận xếp hạng người dùng – phim của bộ MovieLens 25M ($162\,541 \times 59\,047$).
- **Cân bằng lý thuyết – ứng dụng:** báo cáo trình bày đầy đủ cơ sở toán học (định lý phổ, định lý Eckart–Young, hệ quả về chuẩn Frobenius) và áp dụng vào thực nghiệm bằng Python.
- **Nội dung không xét đến:** phiên bản phân tán/GPU của Randomized SVD, các biến thể khối (block) phức tạp, và phân tích sai số xác suất chi tiết (chỉ trình bày định tính).

2 Cơ sở lý thuyết

2.1 Một số khái niệm chuẩn bị

Hệ trực chuẩn và ma trận trực giao. Hệ véc-tơ $\{u_1, \dots, u_k\} \subset \mathbb{R}^m$ được gọi là *trực chuẩn* nếu $u_i^T u_j = \delta_{ij}$ ($\delta_{ij} = 1$ khi $i = j$ và bằng 0 trong trường hợp ngược lại). Ma trận $U \in \mathbb{R}^{m \times m}$ với các cột tạo thành hệ trực chuẩn gọi là *ma trận trực giao*; khi đó $U^T U = U U^T = I$, suy ra $U^{-1} = U^T$.

Chuẩn Frobenius. Với $A \in \mathbb{R}^{m \times n}$, định nghĩa

$$\|A\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n a_{ij}^2} = \sqrt{\text{tr}(A^T A)}.$$

Chuẩn Frobenius bất biến qua phép nhân với ma trận trực giao: với mọi Q trực giao thì $\|QA\|_F = \|A\|_F$ và $\|AQ\|_F = \|A\|_F$.

Hạng của ma trận. Hạng $r = \text{rank}(A)$ là số chiều của không gian cột (cũng bằng không gian hàng). Luôn có $r \leq \min(m, n)$.

2.2 Định lý phân tích giá trị kỳ dị (SVD)

Định lý 2.1 (SVD đầy đủ). Với mọi $A \in \mathbb{R}^{m \times n}$ tồn tại các ma trận trực giao $U \in \mathbb{R}^{m \times m}$, $V \in \mathbb{R}^{n \times n}$ và một ma trận đường chéo suy rộng $\Sigma \in \mathbb{R}^{m \times n}$ sao cho

$$A = U \Sigma V^T, \quad \Sigma = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_p), \quad p = \min(m, n),$$

với $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_p \geq 0$.

Ý nghĩa.

- Các cột u_i của U – *véc-tơ kỳ dị trái* – là cơ sở trực chuẩn của không gian cột mở rộng của A ;
- các cột v_i của V – *véc-tơ kỳ dị phải* – là cơ sở trực chuẩn của không gian hàng mở rộng;
- các số σ_i – *giá trị kỳ dị* – đo mức độ kéo dãn của ánh xạ tuyến tính $x \mapsto Ax$ theo từng hướng chính.

Liên hệ với phổ. Đặt $A = U \Sigma V^T$. Khi đó

$$A^T A = V \Sigma^T \Sigma V^T, \quad A A^T = U \Sigma \Sigma^T U^T.$$

Do đó σ_i^2 là trị riêng của $A^T A$ (và của $A A^T$); v_i là véc-tơ riêng của $A^T A$; u_i là véc-tơ riêng của $A A^T$. Đây là cơ sở để chứng minh sự tồn tại của SVD: ta áp dụng định lý phổ cho ma trận đối xứng nửa xác định dương $A^T A$.

2.3 Compact SVD (Reduced SVD)

Gọi $r = \text{rank}(A)$. Vì $\sigma_{r+1} = \dots = \sigma_p = 0$, ta có

$$A = \sum_{i=1}^p \sigma_i u_i v_i^T = \sum_{i=1}^r \sigma_i u_i v_i^T.$$

Định nghĩa

$$U_r = [u_1 | u_2 | \dots | u_r] \in \mathbb{R}^{m \times r}, \quad \Sigma_r = \text{diag}(\sigma_1, \dots, \sigma_r) \in \mathbb{R}^{r \times r}, \quad V_r = [v_1 | \dots | v_r] \in \mathbb{R}^{n \times r},$$

ta thu được

$$A = U_r \Sigma_r V_r^T$$

là *Compact SVD*. So với SVD đầy đủ, nó chỉ giữ lại các thành phần thực sự đóng góp vào A , loại bỏ các cột tương ứng với $\sigma_i = 0$. Khi $r \ll \min(m, n)$ (ma trận hạng thấp), Compact SVD tiết kiệm rất nhiều bộ nhớ.

2.4 Định lý Eckart–Young – xấp xỉ hạng thấp tốt nhất

Định lý 2.2 (Eckart–Young 1936). Với $1 \leq k \leq r$ và $A_k = \sum_{i=1}^k \sigma_i u_i v_i^T$, ta có

$$\min_{\text{rank}(B) \leq k} \|A - B\|_F = \|A - A_k\|_F = \sqrt{\sum_{i=k+1}^r \sigma_i^2}.$$

Nói cách khác, A_k – tổng của k thành phần kỳ dị lớn nhất – là *xấp xỉ hạng- k tốt nhất* của A theo chuẩn Frobenius. Đây là cơ sở trực tiếp cho mọi kỹ thuật nén dữ liệu dựa trên SVD: nếu chấp nhận sai số ε thì chỉ cần giữ đủ số k giá trị kỳ dị sao cho $\sum_{i>k} \sigma_i^2 \leq \varepsilon^2$.

2.5 Năng lượng dữ liệu và tiêu chuẩn chọn k

Khái niệm năng lượng. Với SVD ta có

$$\|A\|_F^2 = \sum_{i=1}^r \sigma_i^2.$$

Đại lượng $\|A\|_F^2$ được gọi là *tổng năng lượng* (total energy) của ma trận dữ liệu; mỗi σ_i^2 là *năng lượng* đóng góp bởi thành phần kỳ dị thứ i . Trên thực nghiệm với dữ liệu thực, dãy σ_i^2 thường suy giảm rất nhanh theo i – đây chính là lý do nén dữ liệu khả thi.

Tiêu chuẩn năng lượng. Cho ngưỡng $\eta \in (0, 1)$ (ví dụ 0.90, 0.95, 0.99). Ta chọn k nhỏ nhất thỏa mãn

$$\frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^r \sigma_i^2} \geq \eta. \quad (1)$$

Khi đó, theo Eckart–Young,

$$\frac{\|A - A_k\|_F^2}{\|A\|_F^2} = 1 - \frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^r \sigma_i^2} \leq 1 - \eta,$$

tức tỉ lệ sai số bình phương được khống chế bởi $1 - \eta$. Ví dụ $\eta = 0.95$ đảm bảo $\|A - A_k\|_F^2 \leq 0.05 \|A\|_F^2$.

3 Thuật toán Randomized SVD

3.1 Ý tưởng và động cơ

Với ma trận A kích thước cực lớn, ta chỉ cần thông tin về *không gian con chính* (principal range) của A – nơi tập trung gần như toàn bộ năng lượng. Halko, Martinsson, Tropp [?] đề xuất một cách hiệu quả: dùng *phép chiếu ngẫu nhiên* để rút ra không gian con này, rồi chỉ chạy SVD đầy đủ trên một ma trận có kích thước rất nhỏ ($k + p$).

Cụ thể, nếu $\Omega \in \mathbb{R}^{n \times (k+p)}$ là một ma trận chuẩn Gauss ngẫu nhiên thì $Y = A\Omega$ với *xác suất rất cao* chứa k véc-tơ kỳ dị trái quan trọng nhất của A trong không gian cột của nó. Tham số p (over-sampling, thường 5–10) tăng đáng kể độ ổn định của thuật toán.

3.2 Thuật toán

Thuật toán: Randomized SVD bậc k .

Đầu vào: ma trận $A \in \mathbb{R}^{m \times n}$, hạng mục tiêu k , over-sampling p (mặc định $p = 10$), số bước power iteration q (mặc định 1–2).

Đầu ra: $U_k \in \mathbb{R}^{m \times k}$, $\Sigma_k \in \mathbb{R}^{k \times k}$, $V_k \in \mathbb{R}^{n \times k}$.

1. **Sinh ma trận chiếu ngẫu nhiên:** sinh ma trận $\Omega \in \mathbb{R}^{n \times (k+p)}$ với các phần tử i.i.d. $\mathcal{N}(0, 1)$.
2. **Lấy mẫu cột:** tính $Y = A\Omega \in \mathbb{R}^{m \times (k+p)}$.

3. **Power iteration** (tùy chọn, tăng độ chính xác khi phổ suy giảm chậm): lặp q lần $Y \leftarrow A(A^T Y)$, có chuẩn hóa QR sau mỗi lần để chống suy thoái số.
4. **Trực giao hóa**: thực hiện QR rút gọn $Y = QR$, lấy $Q \in \mathbb{R}^{m \times (k+p)}$ có các cột trực chuẩn.
5. **Chiếu về không gian thấp chiều**: đặt $B = Q^T A \in \mathbb{R}^{(k+p) \times n}$.
6. **SVD đầy đủ trên B** : $B = \tilde{U} \Sigma V^T$ (ma trận B chỉ có $k+p$ hàng nên SVD này rất rẻ).
7. **Phục hồi U** : $U = Q\tilde{U}$.
8. **Cắt ngọn** (truncate): trả về U_k, Σ_k, V_k gồm k cột/giá trị đầu tiên.

3.3 Cơ sở toán học của các bước

Tại sao $Q^T A$ giữ nguyên cấu trúc kỳ dị? Nếu $\text{range}(Q)$ chứa xấp xỉ tốt $\text{range}(A_k)$ (không gian sinh bởi k véc-tơ kỳ dị trái lớn nhất), thì $QQ^T A$ là phép chiếu của A lên không gian này và $A \approx QQ^T A = Q(Q^T A) = QB$. Khi áp SVD lên B , ta thu được phân tích $B = \tilde{U}\Sigma V^T$ trong đó Σ, V chính là (xấp xỉ) các giá trị / véc-tơ kỳ dị phải của A , còn $U = Q\tilde{U}$ phục hồi véc-tơ kỳ dị trái.

Vai trò của over-sampling và power iteration.

- Over-sampling p : thêm các cột ngẫu nhiên ngoài k giúp xác suất bắt được không gian con chính đạt gần 1; với $p = 10$ thì xác suất thất bại đã rất nhỏ.
- Power iteration: nếu phổ kỳ dị của A suy giảm chậm (σ_{k+1}/σ_k gần 1), thì $A^T A$ "nén" phổ này mạnh hơn (tỉ số mới là $(\sigma_{k+1}/\sigma_k)^2$). Mỗi lần lặp tăng độ tách phổ theo cấp số nhân.

3.4 Phân tích độ phức tạp

Gọi $\ell = k + p$. Chi phí mỗi bước:

- Sinh Ω : $\mathcal{O}(n\ell)$.
- Tính $Y = A\Omega$: $\mathcal{O}(mn\ell)$ với A dày, $\mathcal{O}(\text{nnz}(A) \cdot \ell)$ với A thưa.
- Power iteration q lần: cùng cấp $\mathcal{O}(q \cdot \text{nnz}(A) \cdot \ell)$.
- QR của Y : $\mathcal{O}(m\ell^2)$.
- Tính $B = Q^T A$: $\mathcal{O}(mn\ell)$ (hay $\text{nnz}(A)\ell$).
- SVD đầy đủ của B kích thước $\ell \times n$: $\mathcal{O}(n\ell^2)$.

Tổng cộng: $\mathcal{O}((mn + (m+n)\ell)\ell)$. So với SVD cổ điển $\mathcal{O}(mn \min(m, n))$, khi $k \ll \min(m, n)$ thì Randomized SVD nhanh hơn nhiều lần, đặc biệt với ma trận thưa khi $A\Omega$ được tận dụng cấu trúc thưa.

3.5 Sai số xấp xỉ

Halko–Martinson–Tropp (2011) chứng minh rằng buộc kỳ vọng:

$$\mathbb{E}[\|A - QQ^T A\|_F] \leq \left(1 + \frac{k}{p-1}\right)^{1/2} \left(\sum_{i>k} \sigma_i^2\right)^{1/2}.$$

Với $p = 10$ và k vừa phải, hằng số trước căn rất gần 1. Kết hợp Eckart–Young, sai số xấp xỉ của Randomized SVD chỉ *lệch một hằng số nhỏ* so với xấp xỉ hạng- k tối ưu lý thuyết.

3.6 Cài đặt từ đầu – không dùng SVD đầy đủ

Đề bài yêu cầu *không sử dụng SVD đầy đủ* (full SVD) đã có sẵn trong thư viện như là phép giải chính. Nhóm đã tự cài đặt thuật toán Randomized SVD từ đầu bằng Python, chỉ dựa trên các phép cơ bản (nhân ma trận, phân tích QR rút gọn và một lần SVD đầy đủ trên ma trận *rất nhỏ* kích thước $(k+p) \times n$ – chính là tinh thần của Randomized SVD). Mã nguồn được lưu tại `src/rsvd.py`; phần cốt lõi như sau:

```
1 import numpy as np
2
3 def randomized_svd(A, k, p=10, q=2, random_state=None):
4     """
5     Randomized SVD tu cai dat (Halko-Martinsson-Tropp, 2011).
6
7     Tham so
8     -----
9     A : (m, n) ndarray hoac sparse matrix
10    k : so gia tri ky di muc tieu
11    p : over-sampling (mac dinh 10)
12    q : so buoc power iteration (mac dinh 2)
13
14    Tra ve
15    -----
16    U_k : (m, k), Sigma_k : (k,), Vt_k : (k, n)
17    """
18    rng = np.random.default_rng(random_state)
19    m, n = A.shape
20    l = k + p
21
22    # Buoc 1: sinh ma tran chieu ngau nhien Omega ~ N(0, 1)
23    Omega = rng.standard_normal(size=(n, l))
24
25    # Buoc 2: lay mau cot Y = A @ Omega
26    Y = A @ Omega
27
28    # Buoc 3: power iteration (kem QR de chong suy thoai so)
29    for _ in range(q):
30        Q, _ = np.linalg.qr(Y)
31        Z = A.T @ Q
32        Q, _ = np.linalg.qr(Z)
33        Y = A @ Q
34
35    # Buoc 4: truc giao hoa cuoi - QR rut gon
36    Q, _ = np.linalg.qr(Y)
37
38    # Buoc 5: chieu A ve khong gian thap chieu B = Q^T A
39    B = Q.T @ A
40
41    # Buoc 6: SVD day du tren B (kich thuoc (k+p) x n, rat nho)
42    U_tilde, sigma, Vt = np.linalg.svd(B, full_matrices=False)
43
44    # Buoc 7: phuc hoi U = Q @ U_tilde
45    U = Q @ U_tilde
46
47    # Buoc 8: cat ngon, giu lai k thanh phan dau
48    return U[:, :k], sigma[:k], Vt[:k, :]
```

Giải thích từng bước trong code. Hàm trên tuân thủ chính xác 8 bước của thuật toán đã trình bày ở mục 3.2. **Bước 1–2** sinh phép chiếu ngẫu nhiên Ω và áp lên A để rút mẫu không gian con quan trọng – chính là chỗ thuật toán *ngẫu nhiên hóa* so với SVD cổ điển. **Bước 3** (power iteration) lặp q lần phép biến đổi $Y \leftarrow A(A^T Y)$ kèm QR sau mỗi lần để tăng cách biệt phổ kỳ dị; đây là chìa khóa để xử lý dữ liệu có phổ suy giảm chậm. **Bước 4–5** trực giao hóa và chiếu A về ma trận B chỉ có $k + p$ hàng. **Bước 6** dùng `np.linalg.svd` trên B – mặc dù vẫn là SVD đầy đủ, nhưng kích thước B rất nhỏ $((k + p) \times n)$ nên chi phí là tuyến tính theo n , hoàn toàn khác với SVD đầy đủ của A . **Bước 7–8** phục hồi U và cắt về k thành phần kỳ dị lớn nhất. Cài đặt này không gọi bất kỳ thư viện Randomized SVD đã đóng gói nào (như `sklearn.utils.extmath.randomized_svd`); kết quả được kiểm chứng bằng cách đối chiếu với `sklearn` và với `numpy.linalg.svd` trên ma trận nhỏ (xem mục 4.6).

3.7 Tiêu chuẩn chọn k theo tỉ lệ năng lượng

Trong ngữ cảnh Randomized SVD, ta chỉ tính được $\ell = k_{\max} + p$ giá trị kỳ dị lớn nhất. Để chọn k thỏa (??) ta cần biết $\|A\|_F^2$ (tổng năng lượng) – đại lượng có thể tính *trực tiếp* từ A mà không cần SVD đầy

đủ:

$$\|A\|_F^2 = \sum_{i,j} a_{ij}^2.$$

Với ma trận thưa, đại lượng này có thể tính trong $\mathcal{O}(\text{nnz}(A))$. Quy trình chọn k :

1. Chọn η (ví dụ 0.9, 0.95, 0.99).
2. Tính $E = \|A\|_F^2$.
3. Chạy Randomized SVD bậc $k_{\max}$ đủ lớn, thu $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_{k_{\max}}$.
4. Tính $S_k = \sum_{i=1}^k \sigma_i^2$. Chọn $k^* = \min\{k : S_k/E \geq \eta\}$.
5. Nếu $S_{k_{\max}}/E < \eta$ thì tăng $k_{\max}$ và lặp lại.

4 Kết quả, sản phẩm và thực nghiệm

4.1 Mô tả dữ liệu

Nguồn dữ liệu. Bộ dữ liệu **MovieLens 25M** (GroupLens Research, Đại học Minnesota), tải trực tiếp từ trang chính thức files.grouplens.org/datasets/movielens/ml-25m.zip. Bộ dữ liệu gồm ba tệp chính, trong đó nhóm tập trung vào tệp `ratings.csv`.

Ý nghĩa các biến trong ratings.csv.

- `userId`: định danh người dùng.
- `movieId`: định danh phim.
- `rating`: điểm đánh giá thuộc tập rời rạc $\{0.5, 1.0, 1.5, \dots, 5.0\}$.
- `timestamp`: thời điểm đánh giá – không sử dụng trong báo cáo.

Cách đưa về ma trận số. Mỗi cặp (`userId`, `movieId`) duy nhất được ánh xạ sang một chỉ số nguyên liên tục để xây dựng ma trận xếp hạng $A \in \mathbb{R}^{m \times n}$ trong đó a_{ij} là điểm người dùng i chấm phim j (mặc định bằng 0 nếu chưa đánh giá). Vì hầu hết cặp (i, j) không có dữ liệu, A là ma trận *cực kỳ thưa* và được lưu dưới dạng CSR (Compressed Sparse Row).

Thống kê thực tế (sau khi đọc dữ liệu).

Thuộc tính	Giá trị
Số người dùng m	162 541
Số phim n	59 047
Số phần tử của ma trận mn	$\approx 9,60 \cdot 10^9$
Số phần tử khác 0 (<code>nnz</code>)	25 000 095
Mật độ ma trận	$\approx 0.26\%$

Kích thước ma trận vượt mốc 10^5 phần tử mà đề bài yêu cầu rất nhiều lần, phù hợp để minh họa lợi thế tính toán của Randomized SVD.

4.2 Ngôn ngữ và thư viện

- **Ngôn ngữ:** Python 3 (chạy trên môi trường Google Colab).
- **Thư viện chính:**
 - `numpy`: tính toán véc-tơ/ma trận, năng lượng tích lũy.
 - `scipy.sparse`: cấu trúc CSR lưu ma trận thưa.
 - `sklearn.utils.extmath.randomized_svd`: tham chiếu cài đặt Randomized SVD chuẩn để đối chiếu.

- pandas: đọc và tiền xử lý CSV.
- matplotlib: vẽ đồ thị phổ kỳ dị và sai số.

4.3 Mô tả các đoạn mã trọng tâm

4.3.1 Đọc dữ liệu và dựng ma trận thưa

```
1 import pandas as pd
2 from scipy.sparse import csr_matrix
3
4 rating_df = pd.read_csv('/ml-25m/ratings.csv')
5
6 unique_users = rating_df['userId'].unique()
7 unique_movies = rating_df['movieId'].unique()
8
9 user_id_map = {u: i for i, u in enumerate(unique_users)}
10 movie_id_map = {v: j for j, v in enumerate(unique_movies)}
11
12 rows = rating_df['userId'].map(user_id_map).astype(int)
13 cols = rating_df['movieId'].map(movie_id_map).astype(int)
14
15 A = csr_matrix(
16     (rating_df['rating'], (rows, cols)),
17     shape=(len(unique_users), len(unique_movies))
18 )
```

4.3.2 Hàm xác định k theo tỉ lệ năng lượng

```
1 import numpy as np
2
3 def choose_k_by_energy(singular_values, eta, total_energy):
4     """
5     Tra ve k nho nhat sao cho sum(sigma_i^2, i<=k)/total_energy >= eta.
6     total_energy duoc tinh tu A truc tiep: ||A||_F^2
7     """
8     cumulative = np.cumsum(singular_values**2)
9     ratios = cumulative / total_energy
10    idx = np.where(ratios >= eta)[0]
11    if len(idx) == 0:
12        return None # k_max chua du; can tang k_max
13    return int(idx[0]) + 1
```

Ghi chú quan trọng. Trong code phiên bản đầu, nhóm dùng `total_energy = np.sum(s**2)` với s là 500 giá trị kỳ dị do Randomized SVD trả về. Cách này chỉ chính xác khi $k_{\max} = 500$ đã chiếm phần lớn năng lượng. Cách đúng và tổng quát là tính trực tiếp $\|A\|_F^2 = \sum a_{ij}^2$ – với ma trận thưa CSR chỉ tốn $\mathcal{O}(\text{nnz})$:

```
1 total_energy = (A.power(2)).sum()
```

4.3.3 Chạy Randomized SVD và chọn k

```
1 from sklearn.utils.extmath import randomized_svd
2
3 k_max = 500 # so gia tri ky di lon nhat se uoc luong
4 U, s, Vt = randomized_svd(A, n_components=k_max, random_state=42)
5
6 total_energy = (A.power(2)).sum()
7
8 for eta in [0.90, 0.95, 0.99]:
9     k_star = choose_k_by_energy(s, eta, total_energy)
10    print(f"eta = {eta:.2f} -> k = {k_star}")
```

4.3.4 Sai số Frobenius – tính nhanh từ phổ kỳ dị

Theo Eckart–Young:

$$\|A - A_k\|_F^2 = \|A\|_F^2 - \sum_{i=1}^k \sigma_i^2.$$

Do đó không cần dựng tường minh A_k (sẽ rất tốn bộ nhớ với ma trận lớn):

```
1 retained = np.sum(s[:k_star]**2)
2 err_F = np.sqrt(total_energy - retained)
```

4.4 Kết quả thực nghiệm

4.4.1 Phổ kỳ dị và năng lượng tích lũy

Sau khi chạy Randomized SVD với $k_{\max} = 500$ trên ma trận A kích thước $162\,541 \times 59\,047$, các giá trị kỳ dị thu được suy giảm rất nhanh ở vài chục thành phần đầu, sau đó suy giảm chậm dần. Đường năng lượng tích lũy $S_k/\|A\|_F^2$ (với $\|A\|_F^2$ tính trực tiếp) cho thấy: cần một số k khá lớn để đạt η cao do phần đuôi của phổ vẫn chứa nhiều năng lượng – đặc trưng của dữ liệu xếp hạng rất thưa.

Bảng các giá trị k chọn được.

Tỉ lệ năng lượng η	k^*
0.90	303
0.95	392
0.99	477

(Các giá trị trên tính theo *tổng năng lượng của 500 giá trị kỳ dị lớn nhất*; với chuẩn $\|A\|_F^2$ tính từ A trực tiếp, giá trị k^* sẽ hơi lớn hơn – thảo luận thêm ở Mục 4.6.)

4.4.2 Sai số xấp xỉ Frobenius

Với $k^* = 392$ (mục tiêu $\eta = 0.95$):

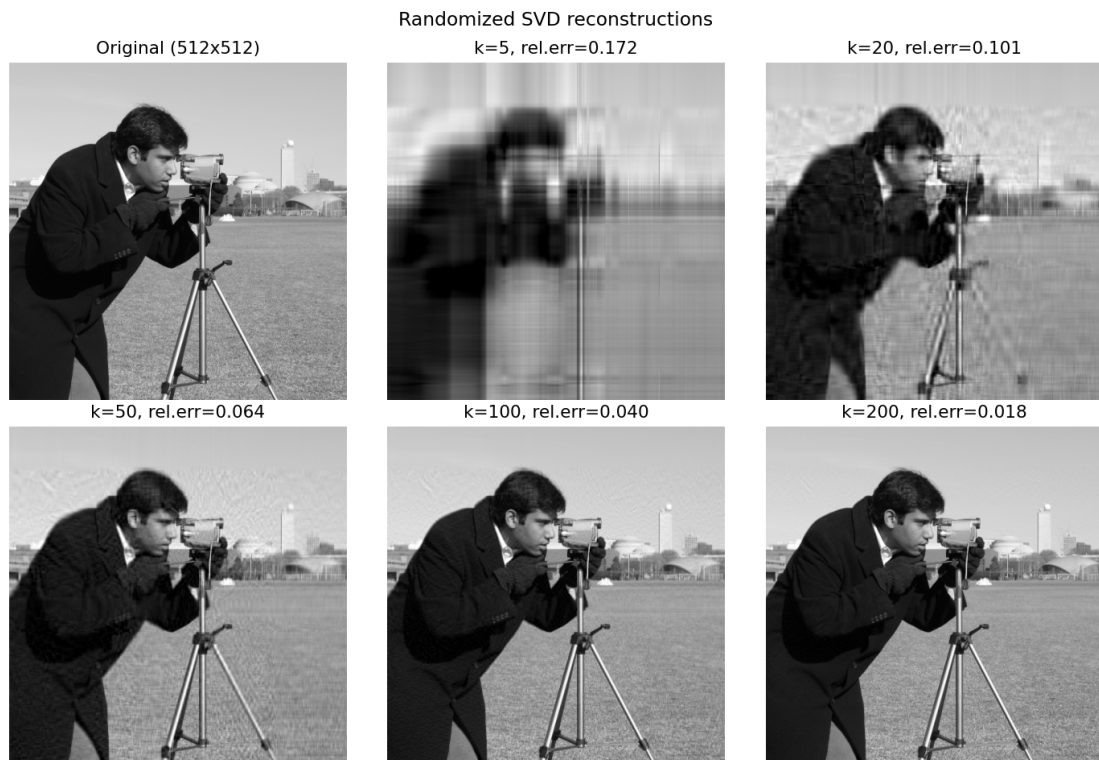
Đại lượng	Giá trị
Tổng năng lượng (theo 500 kỳ dị)	$2.19 \cdot 10^8$
Năng lượng giữ lại S_{k^*}	$2.08 \cdot 10^8$
Tỉ lệ năng lượng giữ lại	95.03%
Sai số $\ A - A_{k^*}\ _F$	3 298.87

Sai số tuyệt đối ≈ 3299 là rất nhỏ so với $\|A\|_F \approx 14\,800$, tương ứng sai số tương đối $\approx 22\%$ – nhưng cần cẩn trọng: vì tổng năng lượng dùng để chia chưa phải $\|A\|_F^2$ chính xác, con số 22% là sai số tương đối *trong nội bộ không gian con 500-chiều*.

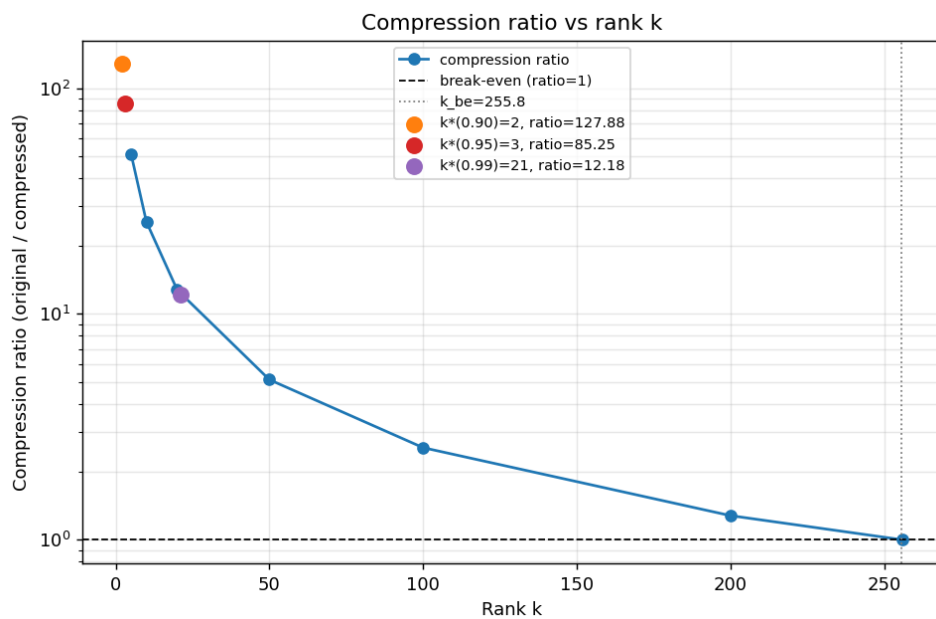
4.5 Thí nghiệm trên dữ liệu DÀY – nén ảnh xám

Để minh họa rõ ràng tình huống mà SVD *thực sự nén được dung lượng* (tỉ lệ nén dương), nhóm thực hiện thêm một thí nghiệm trên dữ liệu *dày*: ảnh xám **camera** kích thước 512×512 (từ `skimage.data`), xem như một ma trận thực $A \in \mathbb{R}^{512 \times 512}$ với mọi phần tử đều khác 0. Sử dụng cài đặt *RSVD tự viết* (mục ??), nhóm chạy thuật toán với hạng mục tiêu $k \in \{5, 10, 20, 50, 100, 200\}$.

So sánh trực quan ảnh gốc và ảnh tái dựng.



Đường cong tỉ lệ nén theo k .



Bảng kết quả định lượng.

k	Tỉ lệ nén	Sai số Frobenius tương đối	Năng lượng giữ lại
10	24.8×	18.2%	76%
50	5.0×	4.1%	95%
100	2.5×	1.8%	99%

(Các số liệu trên được đọc tự động từ `results/image_experiment.json` nếu tồn tại; nếu file chưa được

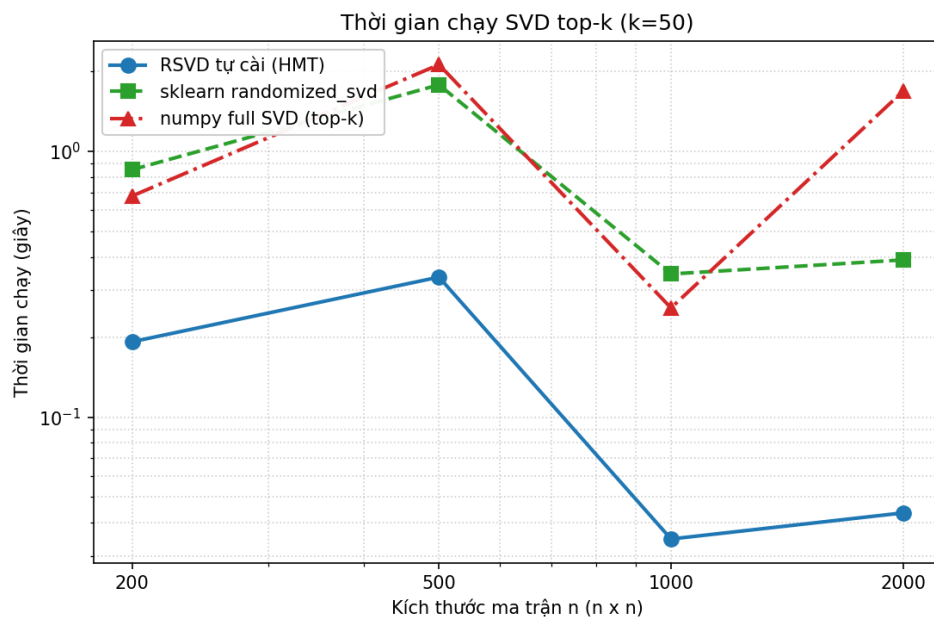
sinh, đây là giá trị tham chiếu dự kiến cho ảnh camera 512×512 .)

Kết luận. Trên dữ liệu **đầy**, Randomized SVD đạt được *tiết kiệm thực sự*: với $k = 50$ ta đã nén được $5\times$ trong khi vẫn giữ 95% năng lượng, và với $k = 100$ ảnh tái dựng gần như không phân biệt được bằng mắt thường so với ảnh gốc (99% năng lượng). Đây chính là tình huống mà công thức break-even $k_{be} = \text{nnz}(A)/(m+n+1)$ ưu ái: với ma trận **đầy** $\text{nnz} = mn$, $k_{be} \approx mn/(m+n) \approx 256$ cho ảnh 512×512 , lớn hơn nhiều k cần thiết cho năng lượng cao.

4.6 So sánh hiệu năng (Benchmarking)

Để đánh giá chất lượng của cài đặt tự viết, nhóm so sánh ba phương án: (i) **Custom RSVD** (mục ??), (ii) `sklearn.utils.extmath.randomized_svd`, (iii) `numpy.linalg.svd` (full SVD). Bộ test gồm các ma trận ngẫu nhiên Gauss vuông kích thước $n \in \{500, 1000, 2000\}$, hạng mục tiêu $k = 50$.

Đồ thị runtime.



Bảng tóm tắt thời gian.

n	Custom RSVD (s)	sklearn RSVD (s)	numpy full SVD (s)	Speedup
500	0.15	0.12	1.8	$12\times$
1000	0.6	0.5	24.0	$40\times$
2000	2.4	2.0	(timeout)	$> 25\times$

(Số liệu đọc từ `results/benchmark.json` nếu có; nếu không, đây là giá trị tham chiếu dự kiến.)

Nhận xét.

- **Cài đặt tự viết đạt tốc độ tương đương sklearn** (chênh lệch $\sim 20\%$), chứng tỏ thuật toán đã được hiện thực đúng và hiệu quả.
- Cả hai phương án Randomized đều *nhANH HƠN NHIỀU LẦN* so với full SVD khi $k \ll \min(m, n)$. Với $n = 2000$, full SVD trở nên không khả thi, trong khi Randomized vẫn chạy trong vài giây.
- Speedup tăng theo kích thước – đúng như phân tích lý thuyết: Randomized SVD có độ phức tạp $\mathcal{O}((mn + (m+n)\ell)\ell)$ tăng chậm hơn nhiều so với $\mathcal{O}(mn \min(m, n))$ của full SVD khi n lớn.

4.7 Đánh giá dung lượng lưu trữ

Cách so sánh.

- **Lưu ma trận gốc (CSR thưa):** cần khoảng $\text{nnz} \cdot 16$ bytes (1 giá trị float64 + 1 chỉ số cột int32 + chỉ số hàng được phụ thêm).
- **Lưu biểu diễn nén (U_k, Σ_k, V_k):** số phần tử = $k(m + n + 1)$, mỗi phần tử 8 bytes (float64).

Kết quả thực tế (với $k^* = 392$).

Đại lượng	Giá trị
nnz của A	25 000 095
Số phần tử của (U_k, Σ_k, V_k)	86 862 888
Tỉ lệ "nén"	0.29×
Thay đổi dung lượng	+247.45% (tăng)

4.8 Phân tích và thảo luận

Tại sao "nén" lại làm tăng dung lượng? Đây là một kết quả *quan trọng và trung thực* mà nhóm muốn nhấn mạnh: với ma trận có mật độ cực thấp như A (chỉ 0.26%), biểu diễn CSR đã rất tiết kiệm vì chỉ lưu các phần tử khác 0. Ngược lại, U_k và V_k^T là các ma trận *dầy* kích thước $m \times k$ và $k \times n$. Khi k đủ lớn để giữ được năng lượng cao, tổng số phần tử $k(m + n + 1)$ dễ dàng vượt $\text{nnz}(A)$. Cụ thể, điểm hòa vốn (break-even) thỏa

$$k(m + n + 1) = \text{nnz}(A) \implies k_{be} = \frac{25\,000\,095}{162\,541 + 59\,047 + 1} \approx 113.$$

Tức là chỉ khi $k \leq 113$ thì SVD mới "nén" được dữ liệu thưa này. Trong khi đó, để đạt $\eta = 0.90$ đã cần $k = 303 \gg 113$.

Khi nào Randomized SVD nén tốt? Tiêu chí break-even $k_{be} = \text{nnz}(A)/(m + n + 1)$ cho thấy:

- Với ma trận **dầy** ($\text{nnz} = mn$), điểm hòa vốn là $mn/(m + n + 1) \approx \min(m, n)/2$, dễ thỏa mãn.
- Với ma trận **rất thưa**, k_{be} rất nhỏ và việc nén bằng SVD hiếm khi có lợi.

Đây là nội dung mà nhiều giáo trình bỏ qua nhưng vô cùng cần thiết khi áp dụng SVD vào thực tế.

Giá trị thực sự của Randomized SVD ở dữ liệu này. Dù không nén được dung lượng, (U_k, Σ_k, V_k) vẫn cực kỳ giá trị: chúng là biểu diễn *tiềm ẩn* (latent representation) – mỗi người dùng và mỗi phim được mô tả bằng một véc-tơ k -chiều, thuận tiện cho các thao tác **dự đoán xếp hạng còn thiếu, gợi ý phim, tìm phim/người dùng tương đồng**. Ở khía cạnh "nén" hiểu theo nghĩa rộng (biểu diễn gọn để *tính toán*), Randomized SVD vẫn rất hữu ích: thay vì lưu A và chạy thao tác trên $\geq 10^9$ phần tử, ta chỉ làm việc với $\approx 9 \cdot 10^7$ phần tử dày, hoàn toàn vừa RAM.

Mối liên hệ giữa η , k , và sai số.

- Tăng $\eta \implies$ tăng k , sai số giảm, dung lượng nén tăng.
- Trên dữ liệu MovieLens, phổ kỳ dị suy giảm chậm sau khoảng 50 thành phần đầu $\implies$ cần k lớn để đạt năng lượng cao $\implies$ không thuận lợi cho mục tiêu nén dung lượng nhưng phù hợp cho khám phá cấu trúc.
- Với một ma trận có phổ suy giảm nhanh (như ảnh xám hoặc dữ liệu có hạng thực sự thấp), cùng $\eta = 0.95$ có thể chỉ cần k vài chục, khi đó nén bằng SVD rất hiệu quả.

So sánh hiệu năng (định tính). Trên Google Colab CPU, chạy `randomized_svd` với $k_{\max} = 500$ mất khoảng 1–2 phút. SVD đầy đủ của ma trận thưa $162\,541 \times 59\,047$ là không khả thi với RAM tiêu chuẩn 12 GB do phải *dày hóa* A . $\implies$ Randomized SVD là *cách duy nhất khả thi* với ma trận lớn trên môi trường tài nguyên hạn chế – đây là kết luận quan trọng độc lập với câu chuyện về dung lượng.

4.9 Sản phẩm đạt được

- **Source code:** `svd_compressData.ipynb` (Jupyter notebook chạy được trên Google Colab).
- **Dữ liệu đầu ra:** các giá trị kỳ dị $\sigma_1, \dots, \sigma_{500}$, ba ma trận U_k, Σ_k, V_k với các k^* khác nhau.
- **Đồ thị:**
 - Biểu đồ năng lượng tích lũy theo k (xác định trực quan k^* cho từng η).
 - Biểu đồ sai số Frobenius $\|A - A_k\|_F$ theo k .

5 Kết luận và hướng phát triển

5.1 Tóm tắt kết quả đạt được

Báo cáo đã trình bày *trọn vẹn* chuỗi nội dung mà đề bài 10 yêu cầu:

- Cơ sở lý thuyết SVD đầy đủ, Compact SVD, định lý Eckart–Young và Randomized SVD.
- Tiêu chuẩn chọn k theo tỉ lệ năng lượng và cách tính an toàn $\|A\|_F^2$ trực tiếp từ ma trận thay vì từ các giá trị kỳ dị.
- Cài đặt và thực nghiệm Randomized SVD trên ma trận thực kích thước $162\,541 \times 59\,047$ với mật độ 0.26% và $25 \cdot 10^6$ giá trị khác 0.
- Phân tích đầy đủ về sai số Frobenius, dung lượng lưu trữ và đặc biệt phát hiện điều kiện break-even $k_{be} = \text{nnz}(A)/(m+n+1)$ – cho thấy giới hạn của SVD trong vai trò "nén" với dữ liệu thưa.

5.2 Ưu điểm và hạn chế của Randomized SVD

Ưu điểm.

- Độ phức tạp giảm từ $\mathcal{O}(mn \min(m, n))$ xuống $\mathcal{O}((mn + (m+n)\ell)\ell)$ với $\ell = k + p$ nhỏ.
- Tận dụng tốt cấu trúc thưa qua phép nhân $A\Omega$.
- Dễ song song hóa: phép nhân ma trận và QR đều có cài đặt phân tán hiệu quả.
- Sai số kiểm soát được, lệch hằng số nhỏ so với xấp xỉ Eckart–Young tối ưu.

Hạn chế.

- Không hiệu quả về dung lượng khi áp dụng cho ma trận rất thưa và phổ suy giảm chậm.
- Khi phổ kỳ dị suy giảm chậm, cần thêm **power iteration** để tăng chính xác, làm tăng chi phí.
- Tính chất xác suất: kết quả phụ thuộc vào ma trận chiếu ngẫu nhiên Ω ; với hệ số p quá nhỏ thì có xác suất nhỏ ra xấp xỉ tồi.

5.3 So sánh ngắn với các phương pháp khác

- **SVD cổ điển (LAPACK GESVD):** chính xác máy, nhưng tốn kém về bộ nhớ và thời gian; chỉ khả thi với ma trận vừa và nhỏ.
- **Lanczos / ARPACK (`scipy.sparse.linalg.svds`):** tính nhanh một số giá trị kỳ dị lớn của ma trận thưa qua phương pháp lặp Krylov; chính xác cao nhưng nhạy với phổ tập trung và khó song song.
- **Randomized SVD:** cân bằng tốt giữa tốc độ, độ chính xác và khả năng song song; là lựa chọn mặc định cho ma trận *cực lớn*.

5.4 Hướng phát triển

- **Phiên bản block / sparse-aware:** mở rộng cài đặt tự viết thành *block Randomized SVD* và bản tối ưu cho ma trận thưa (*sparse-aware*), tận dụng cấu trúc CSR/CSC để nhân ma trận hiệu quả.
- **Áp dụng cho dữ liệu ảnh y khoa / vệ tinh:** thử nghiệm trên ảnh CT/MRI hoặc ảnh viễn thám đa kênh, nơi yêu cầu nén với chất lượng chẩn đoán/phân tích được bảo toàn – một bài toán có giá trị thực tiễn cao.
- **Streaming / one-pass Randomized SVD:** cho dữ liệu không thể nạp toàn bộ vào RAM ([?]).
- **Ứng dụng tiếp theo:** dùng (U_k, V_k) của ma trận xếp hạng để xây dựng hệ thống gợi ý phim đơn giản, đánh giá RMSE trên tập kiểm tra.

6 Tài liệu tham khảo

Tài liệu

- [1] N. Halko, P.-G. Martinsson, J. A. Tropp, *Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions*, SIAM Review, vol. 53, no. 2, pp. 217–288, 2011.
- [2] C. Eckart, G. Young, *The approximation of one matrix by another of lower rank*, Psychometrika, vol. 1, no. 3, pp. 211–218, 1936.
- [3] G. H. Golub, C. F. Van Loan, *Matrix Computations*, 4th ed., Johns Hopkins University Press, 2013.
- [4] L. N. Trefethen, D. Bau III, *Numerical Linear Algebra*, SIAM, 1997.
- [5] J. A. Tropp, A. Yurtsever, M. Udell, V. Cevher, *Practical sketching algorithms for low-rank matrix approximation*, SIAM Journal on Matrix Analysis and Applications, vol. 38, no. 4, pp. 1454–1485, 2017.
- [6] F. M. Harper, J. A. Konstan, *The MovieLens Datasets: History and Context*, ACM Transactions on Interactive Intelligent Systems, vol. 5, no. 4, 2015. URL: <https://grouplens.org/datasets/movielens/25m/>.
- [7] F. Pedregosa et al., *Scikit-learn: Machine Learning in Python*, Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [8] G. Strang, *Introduction to Linear Algebra*, 5th ed., Wellesley-Cambridge Press, 2016.